# PROJECT REPORT

# ON

## Student Resume Generator


**Submitted by:**
**Shubham Gupta**
**UID: 26MCI10128**

**Submitted to:**
**Prof. Palwinder Kaur Mangat(E9422)**


Submitted in partial fulfilment of the requirements for the degree of

**MASTER OF COMPUTER APPLICATIONS (MCA)**


**CHANDIGARH UNIVERSITY**

University Institute of Computing

Punjab, India

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

# BONAFIDE CERTIFICATE

This is to certify that the project report titled **"Student Resume Generator"** has been prepared as an academic project using Python, Tkinter, and other Python libraries. The project demonstrates graphical user interface (GUI) development, student information management, profile photo uploading and webcam capture, live resume preview, input handling, and PDF document generation.

The application enables students to enter their personal information, educational qualifications, technical skills, projects, work experience, certifications, and career objectives. It provides functionality to upload or capture a profile photograph, preview the resume, modify the entered information, and generate and download a professionally formatted resume in PDF format. The project utilizes Tkinter for the graphical user interface, Pillow and OpenCV for image processing and webcam capture, ReportLab for PDF generation, and PyMuPDF for displaying the resume preview within the application.

This project report is submitted in partial fulfilment of the requirements for the degree of **Master of Computer Applications (MCA)**.

Student Signature: _____ Project Supervisor: _____

Head of Department: _____ Date: _____

# TABLE OF CONTENTS

**Chapter 1: Introduction and Background Study** ..... **PAGE**

- 1.1 Project Overview
- 1.2 Problem Definition
- 1.3 Objectives and Scope
- 1.4 Technologies and Libraries Used
- 1.5 Existing System and Proposed Solution

**Chapter 2: System Requirements and Design** ..... **PAGE**

- 2.1 Hardware and Software Requirements
- 2.2 System Architecture
- 2.3 Data Flow and Program Workflow
- 2.4 Data Structure and Transaction Model
- 2.5 Functional and Non-Functional Requirements

**Chapter 3: Implementation and Analytical Methods** ..... **PAGE**

- 3.1 Program Structure and Object-Oriented Design
- 3.2 Sales Data Generation and Management
- 3.3 Revenue Calculation and Statistical Analysis
- 3.4 Graphical User Interface Development
- 3.5 Data Visualization and Chart Integration
- 3.6 Input Validation and Automatic Refresh

**Chapter 4: Testing, Results and Conclusion** ..... **PAGE**

- 4.1 Testing Methodology
- 4.2 Test Cases and Validation
- 4.3 Results and Analysis
- 4.4 Limitations and Future Scope
- 4.5 Conclusion
- 4.6 Pushing the Project to GitHub
- Reference

# CHAPTER 1: INTRODUCTION AND BACKGROUND STUDY

## 1.1 Project Overview

The **Student Resume Generator** is a desktop application developed using Python to simplify the process of creating professional resumes. It allows students to enter their personal information, educational qualifications, technical skills, projects, work experience, certifications, and career objectives through a graphical user interface.

This project addresses these difficulties by providing a simple application that automatically organizes the entered information into a structured resume. The application also allows users to upload a photograph or capture one using a webcam.

The resume is generated in PDF format using ReportLab. PyMuPDF is used to render the generated PDF page so that the user can preview the resume within the application before downloading it.

## 1.2 Problem Definition

Students frequently need resumes for internships, college placements, job applications, and other professional opportunities. However, creating a well-formatted resume manually may require knowledge of document formatting and design.

Common difficulties include:

- Maintaining consistent formatting throughout the document.
- Arranging personal, educational, and professional information properly.
- Inserting and adjusting a profile photograph.
- Checking the final resume before saving or sharing it.

The proposed system provides a desktop-based solution that collects student information, processes the photograph, generates a formatted PDF, and displays a preview before the final document is saved.

## 1.3 Objectives and Scope

### 1.3.1 Objectives

The main objectives of the Student Resume Generator are:

1. To develop a user-friendly desktop application using Python.
2. To simplify the process of creating student resumes.
3. To collect personal, educational, and professional information through a graphical interface.
4. To provide an option to upload a photograph or capture one using a webcam.
5. To generate a professionally structured resume in PDF format.
6. To reduce the time and effort required for manual resume formatting.

### 1.3.2 Scope of the Project

The application is intended primarily for students, fresh graduates, and job seekers who need a basic resume for academic or professional purposes.

The system supports the entry of personal details, educational qualifications, technical skills, projects, experience, certifications, and career objectives. It also supports photograph uploading, webcam capture, resume preview, and PDF generation.

## 1.4 Technologies and Libraries Used

The project uses Python and several libraries to implement its graphical interface, image processing, PDF generation, and file-handling functionality.

| Technology/Library | Purpose |
|---|---|
| Python | Main programming language |
| Tkinter | Creating the desktop graphical user interface |
| ttk | Providing themed GUI widgets |
| Pillow (PIL) | Loading, resizing, and displaying images |
| OpenCV (cv2) | Accessing the webcam and capturing photographs |
| ReportLab | Generating and formatting PDF resumes |
| PyMuPDF (fitz) | Rendering PDF pages for the preview |
| tempfile | Managing temporary preview files |
| os | Supporting file and operating-system operations |
| shutil | Providing file operations where required |
| pathlib | Supporting path handling where required |
| filedialog | Opening file-selection and save dialogs |
| messagebox | Displaying information and error messages |

## 1.5 Existing System and Proposed Solution

### 1.5.1 Existing System

Traditional resume creation methods generally involve word-processing software or online resume builders. Users may need to arrange sections manually, adjust formatting, insert photographs, and repeatedly check the final document.

### 1.5.2 Proposed Solution

The Student Resume Generator provides an integrated interface for entering information and generating a structured PDF resume.

Users can upload or capture a photograph, preview the generated resume, make corrections, and save the final PDF.

| Existing System | Proposed System |
|---|---|
| Manual formatting may be required | Resume formatting is handled by the application |
| Section arrangement requires manual effort | Uses predefined resume sections |
| Photograph insertion may require separate editing | Supports photo uploading and webcam capture |
| Previewing may require another application | Displays the preview inside the application |
| Corrections may require repeated formatting | Allows editing and refreshing the preview |
| File naming must be entered manually | Suggests a filename based on the student's name |

# CHAPTER 2: SYSTEM REQUIREMENTS AND DESIGN

## 2.1 Hardware and Software Requirements

The application requires a computer capable of running Python and the necessary libraries. A webcam is optional and is required only when the user wants to capture a photograph directly.

### 2.1.1 Hardware Requirements

| Component | Recommended Requirement |
|---|---|
| Processor | Standard modern processor |
| RAM | 4 GB or more recommended |
| Storage | Sufficient space for Python, libraries, and generated PDFs |
| Display | Laptop or desktop monitor |
| Keyboard and Mouse | Required for user interaction |
| Webcam | Optional, for photograph capture |

### 2.1.2 Software Requirements

| Component | Requirement |
|---|---|
| Operating System | Windows or another compatible operating system |
| Programming Language | Python |
| GUI Framework | Tkinter |
| Image Processing | Pillow and OpenCV |
| PDF Generation | ReportLab |
| PDF Rendering | PyMuPDF |
| Code Editor | Visual Studio Code, PyCharm, or another Python editor |

The required libraries must be installed before running the application. Webcam functionality also depends on the availability of a working camera and appropriate system permissions.

## 2.2 System Architecture

The application follows a modular structure in which different components perform specific tasks. The graphical interface collects information, the processing logic organizes it, and the relevant libraries handle photographs and PDF operations.

### 2.2.1 Main Components

**1. User Interface Layer**

This layer is developed using Tkinter and ttk. It contains the input fields, buttons, labels, and preview area.

**2. Input Processing Layer**

This layer collects the values entered by the user and prepares them for resume generation. It also performs the implemented full-name check before the final PDF is downloaded.

**3. Image Processing Layer**

Pillow handles image loading and display, while OpenCV provides webcam access and photograph capture.

**4. PDF Generation Layer**

ReportLab generates the resume document using the student's information and the formatting rules defined in the program.

**5. Preview Layer**

PyMuPDF renders the generated PDF page into an image. The image is then displayed in the Tkinter preview area.

**6. File Management Layer**

This layer supports temporary preview generation, saving the final PDF, and cleaning up tracked temporary files when the application closes.

## 2.3 Data Flow and Program Workflow

The application follows a sequence of operations, beginning with information entry and ending with the creation of the final PDF resume.

### 2.3.1 Program Workflow

- The user launches the application.
- The main window appears.
- The user enters personal and academic details.
- The user uploads a photograph or captures one using the webcam.
- The selected photograph is processed and displayed.

- The user clicks the **Refresh Preview** button.
- ReportLab generates a temporary PDF.
- PyMuPDF renders the first PDF page into an image.
- The rendered page is displayed in the preview area.
- The user checks the resume and makes any required changes.
- The user refreshes the preview to check the updated document.
- The user clicks the **Download PDF** button.
- The user selects the save location, and the final PDF is saved.

## 2.4 Student Information Management

The application organizes student information into predefined fields. These values are collected through the graphical interface and used during resume generation.

### 2.4.1 Information Categories

| Category | Details |
|---|---|
| Personal Details | Full name, email, phone number, and address |
| Professional Profile | LinkedIn profile and career objective |
| Education | Course, college, graduation year, and percentage |
| Technical Skills | Programming languages, tools, and other skills |
| Projects | Academic or personal projects |
| Experience | Work experience or fresher status |
| Certifications | Completed courses and certifications |
| Photograph | Uploaded or webcam-captured profile photo |

## 2.5 Functional and Non-Functional Requirements

### 2.5.1 Functional Requirements

Functional requirements describe the operations that the application must perform.

- The application must provide a graphical interface for entering student details.
- The application must allow users to upload a photograph.
- The application must support webcam-based photograph capture.
- The application must generate a resume in PDF format.
- The application must provide a preview of the generated resume.
- The application must allow users to modify information and refresh the preview.
- The application must check that the full name is entered before downloading the final resume.

### 2.5.2 Non-Functional Requirements

Non-functional requirements describe the expected quality and usability of the system.

- **Usability:** The interface should be easy to understand and operate.
- **Performance:** The application should generate and display resume previews within a reasonable time.
- **Reliability:** The application should handle normal user operations without unexpected termination.
- **Maintainability:** The code should be organized so that individual features can be updated.
- **Portability:** The application should run on compatible systems where Python and the required libraries are installed.
- **Privacy:** The current application processes information locally and does not include built-in cloud storage or online profile management.

# CHAPTER 3: IMPLEMENTATION AND APPLICATION DEVELOPMENT

## 3.1 Program Structure and Object-Oriented Design

The application is implemented in a single Python file named main.py. Its main functionality is organized inside the StudentResumeApp class.

### 3.1.1 Main Program Components

| Component | Responsibility |
|---|---|
| Import Statements | Load the libraries required by the application |
| Application Class | Organize the main application functionality |
| GUI Initialization | Create the main window and interface components |
| Input Fields | Collect student information |
| Photo Operations | Upload or capture the profile photograph |
| Preview Operations | Generate and display the resume preview |
| PDF Generation | Create the formatted resume document |
| Download Operation | Save the final resume |
| Cleanup Operation | Remove tracked temporary files when the application closes |

## 3.2 Resume Formatting and PDF Generation

ReportLab is used to create the resume in PDF format. PDF is useful for sharing and printing because it generally preserves the document's layout across compatible PDF viewers.

The application organizes the entered information into predefined resume sections.

### 3.2.1 Resume Sections

**1. Personal and Contact Information**
Contains the student's name and available contact details.

**2. Career Objective**
Presents the student's career objective or professional goal.
**3. Educational Qualifications**
Displays information about the student's course, college, graduation year, and percentage.
**4. Technical Skills**
Lists the student's technical skills and relevant abilities.
**5. Projects**
Describes academic or personal projects entered by the user.
**6. Experience**
Displays the experience information supplied by the student.
**7. Certifications**
Lists the certifications entered by the user.

## 3.4 Graphical User Interface Development

Tkinter is used to develop the desktop interface. It provides the standard widgets needed to collect information and perform application operations.

**Input Form:** Provides fields for personal details, education, skills, projects, experience, certifications, and career objectives.

**Photo Controls:** Allow the user to upload a photograph or capture one using the webcam.

**Refresh Preview Button:** Generates an updated resume preview using the current information.

**Download PDF Button:** Starts the process of saving the final PDF after checking the full-name field.

**Preview Canvas:** Displays the rendered PDF page inside the application.

## 3.5 Photo Uploading and Webcam Integration

The application provides two methods of adding a profile photograph: uploading an existing image and capturing a new image through a webcam.

### 3.5.1 Uploading a Photograph

Pillow loads the selected image and prepares it for display in the application. The selected photograph can then be used during resume generation.

### 3.5.2 Capturing a Photograph Using a Webcam

OpenCV is used to access the webcam and capture an image.

The processed image can then be displayed in the interface and used in the resume.

## 3.6 Resume Preview and Input Validation

The resume preview feature allows users to inspect the generated PDF before downloading it.

### 3.6.1 Input Validation

The current implementation checks whether the full name is provided before the final PDF is downloaded.

The current version does not provide comprehensive email-format or phone-number validation. These checks can be added in future versions.

### 3.6.2 Temporary File Management

The application creates temporary PDF files to display the resume preview. It tracks these files and attempts to remove them when the application closes.

Temporary-file cleanup helps prevent unnecessary preview files from accumulating on the computer.

# CHAPTER 4: TESTING, RESULTS AND CONCLUSION

## 4.1 Testing Methodology

The application should be tested by entering different combinations of student details, uploading photographs, capturing webcam images, refreshing the preview, and downloading the final PDF.

## 4.2 Test Cases and Validation

The following test cases can be used to evaluate the application.

| Test Case | Input or Action | Expected Result |
|---|---|---|
| TC01: Launch Application | Run main.py | Main window opens |
| TC02: Enter Details | Fill in the form | Information is accepted |
| TC03: Upload Photograph | Select a valid image file | Photograph is loaded and displayed |
| TC04: Capture Photograph | Use the webcam option | Photograph is captured if the camera is available |
| TC05: Refresh Preview | Click Refresh Preview | Resume preview is generated and displayed |
| TC06: Edit Information | Modify fields and refresh | Updated information appears in the preview |
| TC07: Missing Full Name | Attempt to download without a name | Validation message is displayed |
| TC08: Download Resume | Enter a name and save the resume | PDF is saved if the operation succeeds |
| TC09: Open PDF | Open the saved document | PDF opens and displays the resume |

| | | |
|---|---|---|
| TC10: Close Application | Close the main window | Application attempts to clean up tracked temporary files |

**Note:** These are proposed test cases. The actual result of each test should be recorded after executing it on the completed application.

## 4.3 Results and Analysis

The form allows users to enter information into predefined fields rather than manually designing the entire document. ReportLab converts the information into a structured PDF resume.

The photograph feature supports image uploading and webcam capture. The preview feature allows users to inspect the generated PDF page before saving the final document.

The ability to edit information and refresh the preview is useful when students need to correct spelling, update skills, change educational details, or modify their career objectives.

## 4.4 Limitations and Future Scope

Although the application provides the main features required for basic resume creation, several improvements can be introduced in future versions.

### 4.4.1 Current Limitations

- **Limited Resume Templates:** The current application uses a predefined layout rather than a large collection of selectable templates.
- **No Database Storage:** The application does not include a database for permanently storing multiple student profiles.
- **Basic Input Validation:** The current implementation checks for the full name but does not comprehensively validate email addresses or phone numbers.
- **Desktop-Based Application:** The application is designed as a desktop program rather than a browser-based service.
- **Library Dependencies:** Python and the required libraries must be installed and configured before the application can run.

### 4.4.2 Future Enhancements

- **Multiple Resume Templates:** Add professional, academic, modern, and creative resume layouts.
- **Database Integration:** Use SQLite or MongoDB to store and retrieve student information.
- **Improved Input Validation:** Add email-format checks, phone-number checks, and clearer error messages.
- **Customizable Design:** Allow users to modify fonts, colours, spacing, and section arrangements.
- **Additional Export Formats:** Consider supporting other document formats where appropriate.
- **Web-Based Version:** Convert the desktop application into a web-based resume builder.

- **AI-Based Suggestions:** Add optional suggestions for improving career objectives, skills descriptions, and project summaries.
- **Multiple Resume Profiles:** Allow users to maintain different resumes for different job roles.

These enhancements could improve the application's flexibility and make it useful to a wider range of users.

## 4.5 Conclusion

Tkinter provides the interface through which users enter their information. Pillow and OpenCV support photograph handling, while ReportLab generates the final PDF. PyMuPDF allows the application to display a preview of the generated PDF page.

One of the main benefits of the project is that users can enter their information, review the resume, make corrections, and download the final document without manually formatting every section in a word processor.

The project also demonstrates the practical use of Python programming, object-oriented design, GUI development, image processing, file handling, and PDF generation.

Overall, the Student Resume Generator provides a useful starting point for automating resume creation. Future enhancements, including multiple templates, improved validation, database integration, and optional AI-based assistance, could make the application more advanced and flexible.

# 4.6 Pushing the Project to GitHub

GitHub is a platform used to store and manage source code through Git version control. Uploading the Student Resume Generator to GitHub can help preserve the source code, track changes, and share the project with faculty members or other developers.

### 4.6.1 Project Folder Structure

A possible project structure is shown below:

Student-Resume-Generator/

- ––|main.py
- ––| requirements.txt
- ––| README.md

The screenshots folder is optional and can contain images of the application interface and resume preview.

The requirements.txt file should list the external Python libraries needed to run the project.

**4.6.2 Github Link :** https://github.com/Shubhamm-9016/Python_Mini_Project.git

# REFERENCES


- Python Software Foundation. *Python Documentation*. https://docs.python.org/3/
- Python Software Foundation. *Tkinter — Python Interface to Tcl/Tk*. https://docs.python.org/3/library/tkinter.html
- Pillow Contributors. *Pillow Documentation*. https://pillow.readthedocs.io/
- OpenCV. *OpenCV Documentation*. https://docs.opencv.org/
- ReportLab. *ReportLab Documentation*. https://docs.reportlab.com/
- PyMuPDF. *PyMuPDF Documentation*. https://pymupdf.readthedocs.io/
- Git. *Git Documentation*. https://git-scm.com/doc
- GitHub. *GitHub Documentation*. https://docs.github.com/